Polynomial Evaluator Accelerator

Team 2: Nicole Baker, Jeremy Bloom, Graham Rogers, Andreas Tzitzikas

ENEE408C Section 0101

Instructor: Prof. Shuvra S. Bhattacharyya

Submission Date: December 15, 2025

Contents

1 Contributions . . . 1

1.1 Nicole Baker . . . 1

1.2 Graham Rogers . . . 2

1.3 Jeremy Bloom . . . 3

1.4 Andreas Tzitzikas . . . 4

2 Executive Summary . . . 5

3 Goals and Design Overview . . . 6

4 C-Specific Design Overview . . . 9

5 Realistic Constraints . . . 11

6 Engineering Standards . . . 13

7 Broader Considerations . . . 14

8 Alternative Designs and Design Choices . . . 15

8.1 Design One . . . 15

8.2 Design Two . . . 16

8.3 Design Three . . . 18

8.4 Design Four . . . 19

8.5 Comparison Between Designs . . . 21

8.6 Pareto Analysis . . . 26

9 Technical Analysis for System and Subsystem . . . 28

10 Design Validation for System and Subsystems . . . 29

11 Test Plan 30
12 Project Planning and Management 31
13 Conclusion 32

1 Contributions

1.1 Nicole Baker

I collaborated with my teammates to help come up with the structure and dataflow of the PEA actor. While my teammates worked on the C code for the PEA Actor I designed the unit tests in parallel to thoroughly test the functionality of the PEA. I designed all of the tests to include edge cases and test all of the error modes. I then wrote the expected errors and correct output associated with each of the tests. I also wrote the driver function for the testbench for testing the C-code and documented all of the tests. After the C code was completed, I worked with my teammates to verify the correct outputs using dxtest and correct the code where needed.

After the Verilog code was completed, I assisted my team with testing the code and verifying the outputs matched with the C code. I then helped brainstorm alternate designs for the Verilog design of the PEA. Building further off of my teammate's design, I created an alternate design using Horner's method for polynomial evaluation. I implemented this in a module in Verilog and verified the output and performance in another test directory using the same test vectors and the simulator available. I then assisted with the synthesis of the final designs.

Lastly, I contributed to the report by writing the Executive Summary, Realistic Constraints, Alternative Designs (Design Two and performance analysis), Test Plan, and Project Planning and Management.

I certify this report and affirm my adherence to the University Honor Pledge.

Signature: Nicole Marie Baker

1.2 Graham Rogers

I spent my initial focus on the project helping to design the overall PEA actor, including creating data-flow tables, state transition charts, and enable/invoke methods. After creating a general design, I spent a lot of time working on the software/C implementation of the PEA, working on both general actor flow as well as specific instruction implementation. I then verified the design against our team's test suite, fixing bugs/discrepancies along the way.

After completing the C portion of the project, I began implementing an alternative Verilog design, specifically by re-designing the instruction load mechanism. This resulted in a design with a slightly decreased Cycles Per Instruction (CPI) rate, demonstrating a positive, potential improvement on our base design. Once this design was completed, I helped my team by performing synthesis and implementation for all designs, as I analyzed all results and comprised relevant charts to detail the differences between all Verilog designs.

In the report, I contributed to the following areas: Goals and Design Overview, Engineering Standards, Alternative Designs and Design Choices, Conclusion. I also helped make descriptive versions of the figures seen dispersed throughout the project.

I certify this report and affirm my adherence to the University Honor Pledge.

Signature: Graham Rogers

1.3 Jeremy Bloom

At the start of this project the team, including me worked together to read through the project requirements, divide up work, and plan the initial structure of our polynomial evaluator accelerator. I then took up work on the invoke_funcs.c file which contains the execution portion of the invoke command for the PEA. I worked with Graham and Nicole in order to organize actor specifications, expected test results, and the design of the context struct. The last step of the C implementation was working to debug and fix issues in the PEA with my efforts focused on bugs in my invoke_funcs.c file.

In the later half of the labs with the C implementation complete, I worked to understand Andreas's Verilog design in order to produce a design of my own. Eventually I came up with a Kogge-Stone adder which was fairly simple to type out. However synthesis and integration proved more challenging with a few bugs in my code and then aggressive Vivado optimizations causing synthesis and implementation to softly fail by not correctly implementing the Kogge-Stone adder while still passing the pre-synthesis tests. I resolved these issues eventually by fighting with Vivado synthesis and implementation optimization algorithms by adding (* keep_hierarchy = "yes" *) and (* dont_touch = "true" *) statements to my adder design file.

I contributed to this design document in various sections such as my design 3 writeup. I also wrote the C-specific design overview and created the accompanying diagram showing the interaction between various C files and the Lide-C library. Finally, I wrote the broader considerations section and added my design_3 data to the graphs and tables.

I certify this report and affirm my adherence to the University Honor Pledge.

Signature: Jeremy Bloom

1.4 Andreas Tzitzikas

I started by working closely with the team to develop the structure and data flow for the C portion of the code. Once the team finished the C model, I began working on the Verilog portion of the project, specifically implementing the initial design, Design 1. I created this design to closely mimic the structure of the C code, where each hardware module translates directly to a function in the C code. This approach allowed for quick development of the Verilog code and enabled us to standardize testing across both Verilog and C. Currently, both the Verilog and C drivers take in the exact same test input files and produce the same output format, allowing for quick comparison and validation of test cases.

Specifically, I worked on developing the data flow and the computation of each command and input data. All Verilog codes for the PEA evaluator were contained within the pea_top module, which combines both the data path and the project controller. The PEA controller used an FSM that had 5 states corresponding to the functional module currently being used. For example, if the system was in the LOAD_INSTRUCTION state, the controller would use the STP module to load data (Figure 1).

In addition, I worked on setting up the synthesis, writing in-depth documentation, and collaborating with the team to find better designs. I helped teammates implement Designs 2, 3, and 4, which were all modifications of my initial design. Building a solid base design early on allowed us to speed up the development and testing of these later designs.

I certify this report and affirm my adherence to the University Honor Pledge.

Signature: Andreas Nikolaos Tzitzikas

2 Executive Summary

This project investigates the design, implementation, and evaluation of the Polynomial Evaluation Accelerator (PEA). The PEA interfaces with four external FIFO buffers that supply control instructions and input data, and receive computed results and status outputs. The PEA performs four operations: storing up to 8 polynomials of up to degree 10 as coefficient vectors (STP), evaluating the result of a polynomial stored at a given address (EVP), evaluating a polynomial stored at a given address b times (EVB), and resetting the array of coefficient vectors to the empty state (RST).

The PEA was implemented in two different environments, one as a software component in C and the other as a hardware component in Verilog. In the software component, the PEA is implemented as a LIDE-C actor, with input FIFOs from the Control and Data buffers, and connected to output FIFOs for the result and status outputs. In the hardware component, the PEA is implemented in Verilog using LIDE-V, integrating dataflow programming in Verilog [**?**]. The group collaborated to design and plan the C and Verilog components of the PEA. We then split up tasks, so that 2 team members began work on implementing the C code, one worked on developing the unit tests independently, and one team member worked on the Verilog implementation, mirroring the C implementation.

The C code was completed first to give an overall guideline for the architecture of the PEA design. The tests were developed in parallel to provide more rigorous tests, so that the code was changed to fit the tests instead of writing the tests around the code. The same test cases were used to test the Verilog implementation.

After writing the initial C and Verilog code, we considered different strategies to reduce the amount of hardware, latency, and power consumption of the PEA Verilog design. We implemented these alternate designs using LIDE-V and used Vivado Design suite to synthesize and implement the designs. We were then able to analyze the generated reports to analyze the performance of our designs, finding that instruction encoding and the algorithm used to compute the polynomial had a large impact on performance.

3 Goals and Design Overview

The goal of this project was to create an effective polynomial evaluator accelerator actor, first as a software implementation using LIDE-C and then in Verilog using lightweight dataflow in LIDE-V, and our secondary goal was to create alternate hardware designs in Verilog to improve the speed of our calculations and decrease the area of our design. We were able to accomplish both of our goals in this project.

The first challenge we addressed in our design was how to design our PEA actor around the commands we were required to implement. We wanted to use a minimal amount of states to decrease complexity; another challenge was determining how to read in tokens specifying commands and data. We designed our PEA actor around the Actor Dataflow table shown in Table 1. We separated our modes into load instruction, and then each of the four commands was a different mode, allowing us to read data and output results and status in the same firing of the PEA. For our load instruction mode, we decided to have the PEA read 3 tokens from the control input corresponding to the instruction op-code and up to two possible arguments depending on the instruction. This was not the most space efficient way to encode instructions in our control inputs, but this format simplified our testing and improved readability and debugging for our user.

Mode	Control Input	Data Input	Result Output	Status Output
Load Instruction	-3	0	0	0
Execute STP	0	-(N+1)	+1	+1
Execute EVP	0	-1	+1	+1
Execute EVB	0	-b	+b	+b
Reset	0	0	0	0

Table 1: PEA Actor Dataflow Table.

The Verilog implementation of the PEA follows a modular architecture using LIDE-V principles, structured as a hierarchical system of nine interconnected modules. The top-level module, `pea_top.v`, serves as the interface between the external FIFO buffers and the internal PEA subsystems, managing data routing and coordinating the execution of the

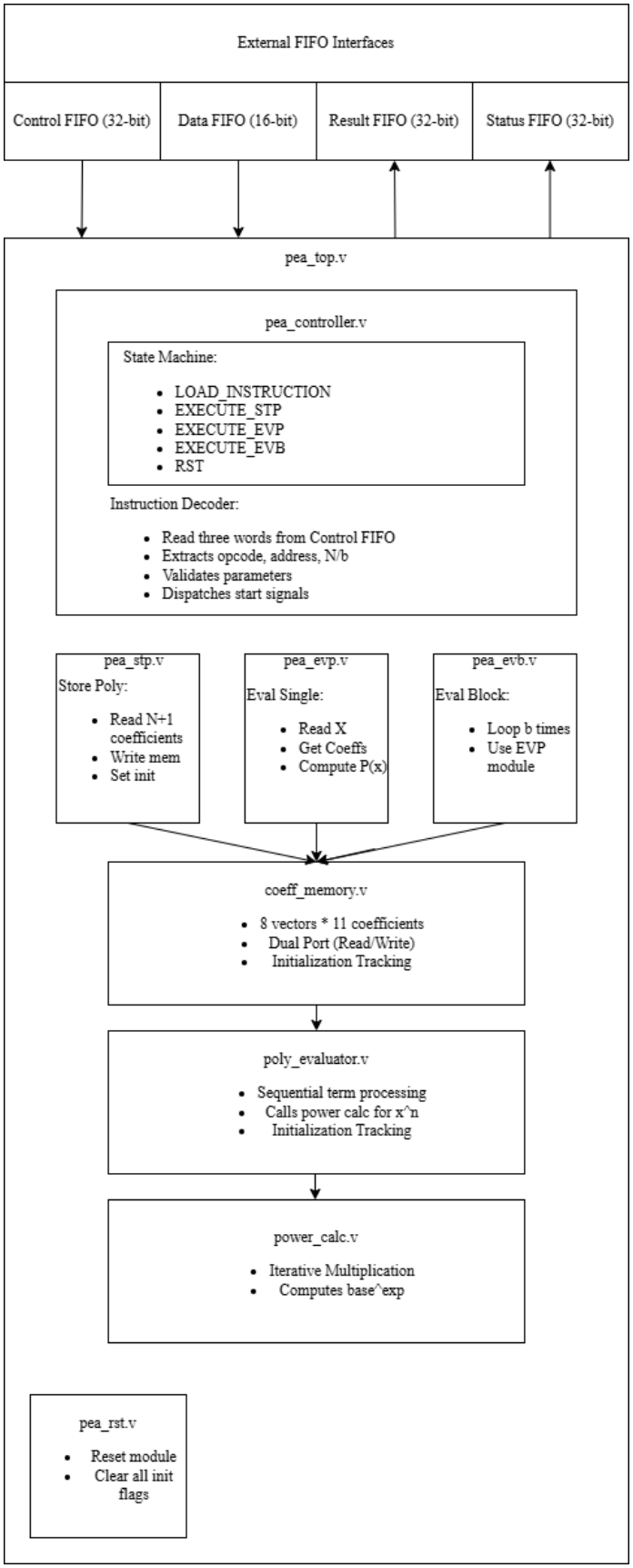

Figure 1: PEA Verilog Implementation

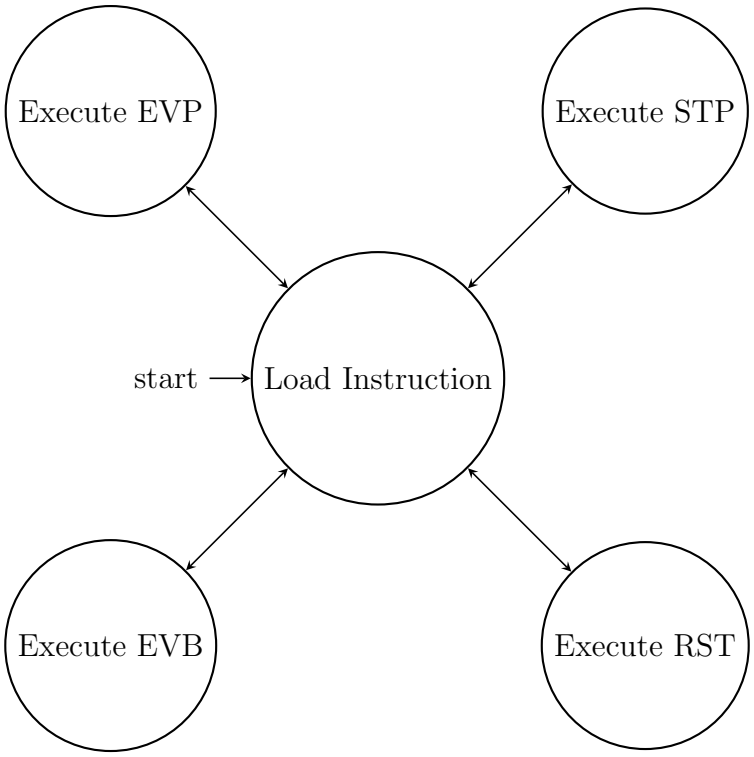

Figure 2: PEA Actor State Flow Diagram

four primary operations. The design employs a centralized controller, `pea_controller.v`, which implements a state machine that reads and decodes instructions from the control FIFO, validates parameters, and dispatches start signals to the appropriate operation modules. Each operation is implemented as a dedicated module: `pea_stp.v` for storing polynomial coefficients, `pea_evp.v` for single polynomial evaluation, `pea_evb.v` for batch evaluation, and `pea_rst.v` for resetting the system. The polynomial evaluation itself is performed by `poly_evaluator.v`, which computes polynomial results term-by-term using a sequential power calculation approach, supported by `power_calc.v` for computing x^n values iteratively. Coefficient storage is managed by `coeff_memory.v`, which maintains eight coefficient vectors, each capable of storing up to eleven 16-bit signed coefficients, with initialization tracking to ensure data integrity. The polynomial evaluator maintains a six-state machine (IDLE, WAIT_POWER, MULTIPLY, ACCUMULATE, NEXT_TERM, DONE) that coordinates with the power calculator, which uses a three-state machine (IDLE, COMPUTE, DONE) to perform iterative multiplication for power computation.

The coefficient memory module implements a dual-port memory structure with separate read and write interfaces, allowing concurrent access for coefficient storage during STP operations and coefficient retrieval during EVP operations. The design employs signed arithmetic throughout, with 16-bit signed coefficients and 32-bit signed results, ensuring correct handling of negative values and preventing overflow in intermediate calculations. Error detection is implemented at multiple levels, with parameter validation in the controller and operation-specific validation in each operation module, providing comprehensive error reporting through the status FIFO interface. The Verilog design emphasizes dataflow principles, where each module communicates through well-defined interfaces using handshaking signals such as `start`, `complete`, and error flags. This modular approach facilitates both design comprehension and future modifications, as each component has a clear, single responsibility. The design implements comprehensive error handling, with validation checks for invalid addresses, uninitialized coefficient vectors, and out-of-range parameters, returning appropriate status codes to the external system. The architecture supports concurrent operation preparation while maintaining sequential execution guarantees, allowing the controller to prepare the next instruction while the current operation completes, thereby improving overall system throughput.

4 C-Specific Design Overview

Our C implementation of the PEA was designed to utilize the same actor flow state and produce the same results. However, the implementation is very much different due to the discrepancy between C and the hardware-focused design of Verilog. The implementation is shown in Figure 3. The C implementation utilizes LIDE-C as the lightweight data flow design library, with two C files defining the PEA actor and one driver file. Driver.c is used to setup the four LIDE-C FIFOs, instantiate the PEA actor and I/O actors, connect actors and FIFOs and finally start the scheduler. Then the LIDE-C library handles reading from

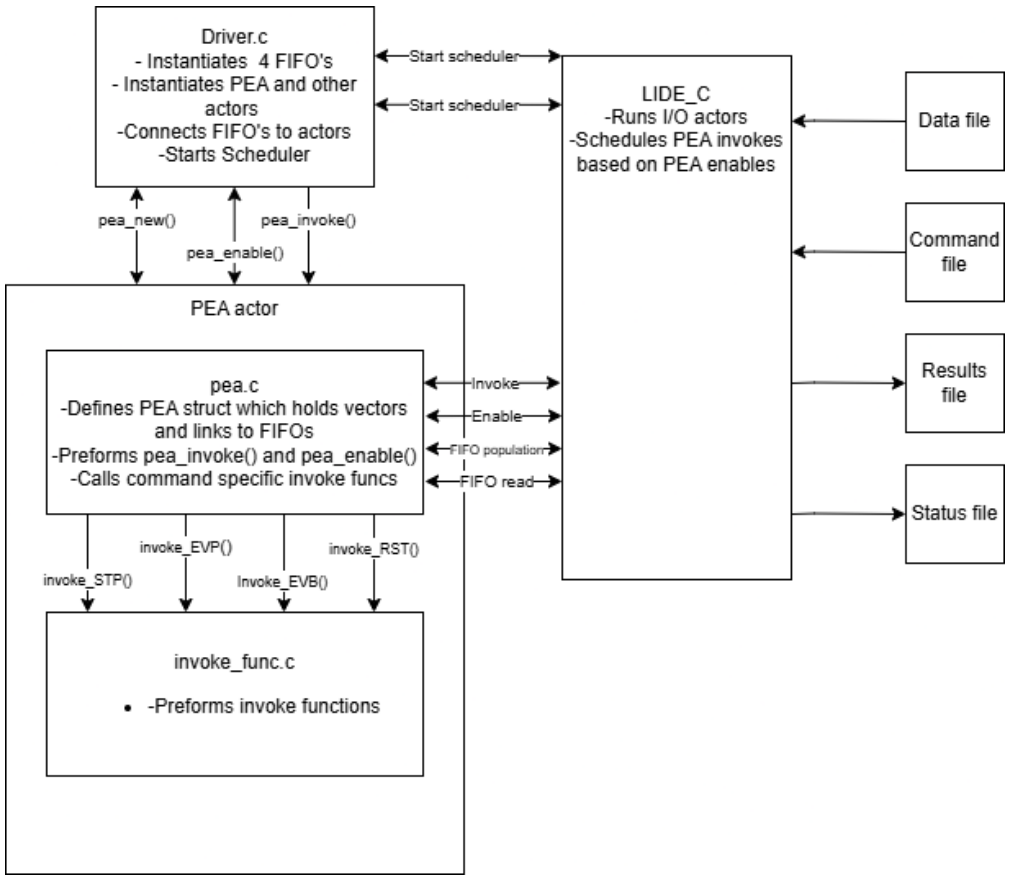

Figure 3: C-implementation diagram

and writing from files, the I/O actors, and most importantly scheduling invokes based off of enable signals.

The PEA actor consists of the file pea.c with invoke_funcs.c providing command-specific invoke functions. First pea.c and its corresponding .h file define the PEA context struct. This struct stores the coefficient vectors and the current mode along with pointer to the PEA's invoke and enable functions, and FIFO pointers. The enable functions queries the LIDE-C population function on each FIFO and determines if there is data and space in FIFO's in order to invoke the PEA actor for different modes. The PEA actor has 5 modes named LOAD_INSTRUCTION, STP, EVP, EVB, and RST.

The PEA actor starts of in the LOAD_INSTRUCTION mode where the enable signal when there is at least 3 data points in the control FIFO. The scheduler will see the enable signal and call pea_invoke() which will run the LOAD_INSTRUCTION case. The

LOAD_INSTRUCTION case will read in an opcode and 2 arguments which can mean different things depending on the opcode. The mode will take on the value of the opcode (STP, EVP, EVB, or RST), and place the arguments in the struct in the coefficient vector address, N value (STP's number of coefficients), or B value (EVB's number of evaluations). Now the enable signal will be be looking for the right data and output space for the particular mode. When these requirements are fulfilled the scheduler will call invoke again which in turn will call the invoke_funcs.c function depending on the mode (which is the read in command). For example, lets follow the invoke_EVP() function which will first check that the coefficient_vector address from the instruction arguments is in bounds and initialized. Next it will read in a single value from the data FIFO and preform the polynomial calculation using the coefficient vector and the new value as the "x" value. Finally the function will write to the result and status FIFOs and the PEA actors mode will be set back to LOAD_INSTRUCTION.

5 Realistic Constraints

In this project, our objective was to create a Polynomial Evaluator Accelerator (PEA), using dataflow programming to implement the PEA as an Actor to integrate with input and output FIFOs. The main constraints on our design are described below.

One of the constraints was the size of input and output tokens that were stored in the FIFOs and read or output by the PEA. The size of the tokens was required to be at least 16 bits for the data input and 32 bits for the result output. There was more flexibility with the size of the control input, but it had to be at least 10 bits to account for the opcode and optional arguments for instructions. Additionally, for the status output FIFO there was a requirement of having a status output for every result output; thus, the size of our token to our status output FIFO had to fit all encoding for success and error handling. In addition, the FIFOs needed to share information about available space and population, in order to

for the enable and invoke functions to work successfully.

The enable and invoke functions in C that were equivalent to the actor invoke and enable modules in Verilog were necessary parts of the Actor module implementation. These modules and functions facilitated the scheduling and firing of each of the modules. The enable modules were necessary for checking the population and empty space in the FIFOs, verifying that enough resources were available for the Actor to fire. The invoke modules were responsible for then calling the functions for each of the modes of the Actor to complete the computations. Thus, the invoke must also make sure that each of the other functions/modules is provided with the proper inputs. This makes the invoke and enable modules a necessary requirement of our actor design to utilize the dataflow model.

In order to implement our PEA actor, we had to use programming and hardware description languages that allowed us to easily implement a dataflow based design. In software, C was the best choice for programming due to the low-level features, and the ability to implement structs and utilize LIDE-C FIFOs. To create our hardware design, it was necessary to use the Verilog hardware description language, which enabled us to cleanly break up our design into modules that we could interconnect to easily implement our final actor. Verilog also simplified simulation, synthesis, and implementation, allowing us to easily test and evaluate our designs.

The testing requirements further added to the constraints on the project. The testing requirements required individual testing of each of the commands individually as well as streams of multiple commands with varying arguments. Testing each of the functions allowed us to test edge cases as well as error modes, testing the maximum and minimum input parameters, as well as incorrect inputs, to verify the functionality of the PEA. The constraints required us to write at least fifteen unique unit tests to validate our design.

Another main constraint that shaped the design of our PEA was the model-based design constraint. This required us to break up our design into four main modes corresponding to each of the commands, as well as a mode to load the instruction. Since we do not know

how much data will need to be processed before reading the instruction, it is necessary to break up the design into different modes in our Actor that are each called independently.

6 Engineering Standards

This project has been guided by the use of well-established engineering standards, particularly the use of C and Verilog as standard programming and Hardware Description Languages (HDLs) respectively.

The most important feature of any modern programming language/HDL is the ability to create program structure and modularity. In C, this is accomplished via the use of functions, which allow programmers to create snippets of code that can be re-used almost anywhere else in a given project. In Verilog, this can be implemented by modules, which can be thought of as the HDL-equivalent version of functions. By using these constructs, we organized our code in a very modular way: each instruction type received its own function/module (e.g. STP, EVP), allowing higher-level control functions/modules to call these and maintain a layer of abstraction/separation. Furthermore, especially in the Verilog portion of the project, techniques such as addition or exponential calculations were broken into their own modules, allowing us to experiment with different types of implementation while leaving the rest of the system un-changed. The result of doing all of this is a modular design that allows for easy development, especially in a team of four people.

Another key feature/standard in Verilog is the ability to describe systems in a behavioral manner, which allows developers to focus on the higher level implementation of a system similar to programming languages like C or Python [**?**]. Writing code in this manner allowed us to focus more on ideas such as instruction/algorithm implementation and the system wide implementation of the PEA as opposed to worrying about implementing lower-level ideas such as flip-flops and logic gates. For example, our main design uses the behavioral addition and multiplicative operators (+ and *) rather than

using structurally designed adders and multipliers, allowing us to quickly implement these operations that would be trivial in software design.

7 Broader Considerations

Hardware acceleration using FPGAs and ASICs is frequently relied upon in mission-critical applications in aerospace, healthcare, and military applications. Reliability is a primary concern in these extremely complex systems, as an unaddressed failure can impact the safety of humans and cause high monetary loss. For example, an accelerator drawing too much power in a fighter jet could compromise the power delivery of another system and cause a crash. Applications in medical devices and infrastructure, also demand reliability as a failure can harm the public health and welfare. While our polynomial Evaluator Accelerator is a purely academic design, we still implemented design choices and engineering practices required in mission-critical applications.

One of the primary safety mechanisms in digital design is robust testing, which we implemented for both our C and Verilog implementations of the PEA. The tests are divided into five categories, four testing a specific instruction and one category using multiple instructions in longer tests. This categorization of tests demonstrates that the testing suite covers all of the functionality of the PEA, with the multi-instruction having the critical role of verifying that instructions can work together without unintended side affects. Other safety mechanisms in our process included using errors to avoid undefined behavior and analyzing hardware use and power consumption to ensure our accelerator acts as expected, and does not compromise other systems in a potential real-world application.

Economic factors are also important with price to performance being critical to an accelerator like our being included in the architecture of a larger system. If an accelerator requires to much FPGA or die space then it would not be economical to include. FPGA's have a limited amount of LUTs and Flip-flops while, greater die space becomes

exponentially more expensive due to lower yields. Therefore we preformed numerous analysis of the power usage, hardware usage, and performance to find which one of our designs is the best for different constraints. For example, design two using Horner's method achieves faster calculations but in turn uses much more hardware. Environmental concerns must also be considered with these type of projects. However the scale of our power and hardware usage of all designs are minimal enough that environmental concerns are a non-factor with so many datacenter and other technology using far greater power and generating much more e-waste.

As for global, social, and cultural factors we were primarily considered with utilizing global engineering standards such as version control (git), data-flow design, and coding standards. These technologies and techniques allow for a global team to all contribute to the same project while also aligning with agreed upon standards for this type of development. Socially and culturally, collaborative design like ours is extremely important in making hardware development accessible to students. Additionally, good collaborative design aids students and engineers in building the professional relationships that are key to succeeding in the job market. In conclusion, our design and engineering practices demonstrates the ability to build a solution while also considering both immediate and broader impacts.

8 Alternative Designs and Design Choices

8.1 Design One

Design One employs a straightforward polynomial evaluation method where each term $a_i \cdot x^i$ is computed sequentially, requiring the power calculator to compute x^i for each term from $i = 0$ to $i = 10$, resulting in a latency of approximately 66 clock cycles for a 10th-degree polynomial evaluation. This approach was chosen as the baseline design for its conceptual clarity and implementation simplicity, prioritizing correctness and

maintainability over raw performance. The design cleanly separates concerns with the polynomial evaluator orchestrating the evaluation process while the dedicated power calculator module handles all power computations, facilitating independent testing and validation. The intuitive state machine structure, with clear states for waiting on power calculations, multiplying coefficients by powers, and accumulating results, establishes a functional baseline with transparent mathematical operations that facilitate verification against the C reference implementation, making design_one an appropriate starting point for design exploration and comparison with alternative implementations.

8.2 Design Two

Design Two reuses much of the finite state machine architecture of design One, with a focus on improving the logic for calculation of the polynomials. The polynomial calculation strategy of Design One, loops through the coefficient vector calling the power_calc module for each coefficient. This is a logical process for calculating polynomials, but is inefficient regarding the number of clock cycles it takes to complete the calculation. In Design Two we replaced this design with a module that evaluated the polynomial using Horner's Method described in figure 4. [**?**]

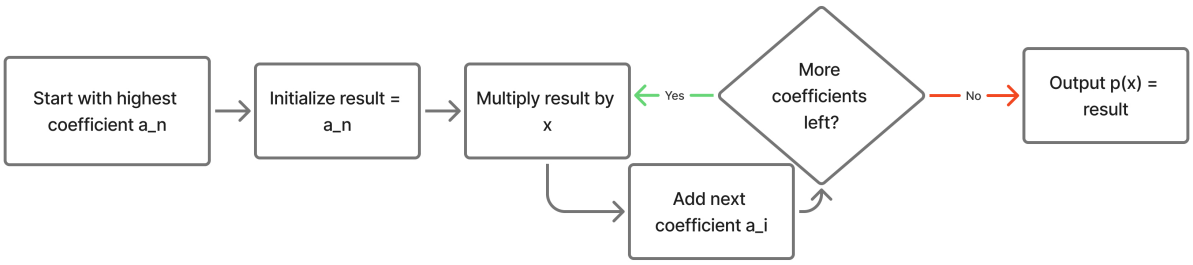

Figure 4: Horner's Method for evaluating Polynomials

Horner's Method for evaluating polynomials rewrites the polynomial in a nested

structure to minimize computation. Instead of calculating powers like x^2, x^3, or x^n separately, the polynomial is reorganized so that each step only requires one multiplication and one addition. For example, a polynomial such as

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \tag{1}$$

is rewritten as

$$P(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_n))) \tag{2}$$

This nested structure allows the polynomial to be evaluated sequentially by starting with the leading coefficient and repeatedly multiplying by x and adding the next coefficient. As a result, Horner's method significantly reduces the total number of operations, making it faster and more numerically stable than direct evaluation.

Each clock cycle, a multiply and add is performed, making the maximum number of clock cycles needed to calculate a polynomial 1 + the order of the polynomial. This is an improvement over the original polynomial evaluator design, where given a degree-n polynomial

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \tag{3}$$

and assuming that computing x^n requires n clock cycles, and the addition is performed in an additional clock cycle, the maximum number of clock cycles needed to evaluate the polynomial using the original method is

$$T(d) = d + \sum_{k=1}^{d} k = \frac{d(d+3)}{2} \tag{4}$$

For the 10th order polynomial, this results in a 54 clock cycle reduction, significantly reducing the latency of polynomial calculation in our design, while also reducing hardware requirements, by removing additional states in our finite state machine to perform the multiply and add in one cycle instead of having a state to wait for the power_calc module

to complete. We also removed the power_calc module itself reducing hardware by removing that finite state machine, decoding logic, and enable and ready inputs associated with it. This design did increase the amount of hardware significantly due to the way that it was implemented in Verilog. The design involved loading in all of the coefficient vectors and storing them, this used more registers to store those data values during the computation. Additionally, the way the multiplication and addition was written in the Horner method verilog code, caused the synthesis to infer a Multiplier out of LUTs instead of using a more efficient multipliers.

8.3 Design Three

Design Three replaces the default behavioral addition with a structural Kogge-stone adder for the 32 bit polynomial additions (shown in Figure 5). As a member of the carry-look-ahead adder family, a Kogge-Stone adder uses internal propagate and generate bits to "pass along" and "create" a carry signal up the bits. These propagate and generate signals are combined using increasingly long spans as shown in the image below.

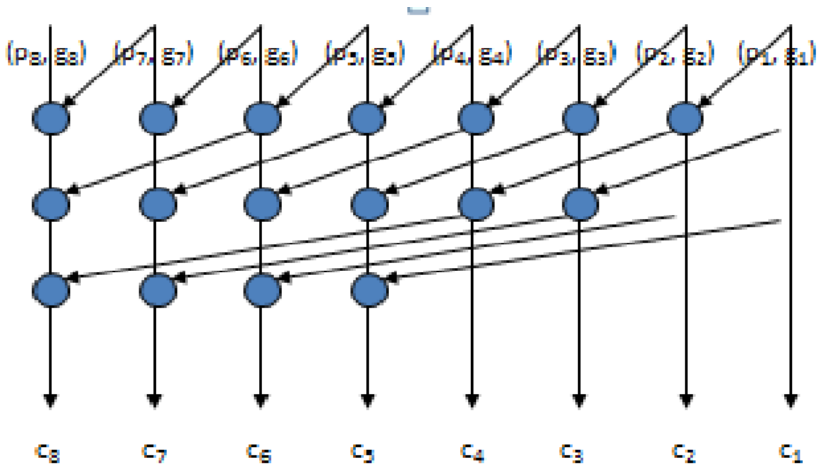

Figure 5: 8 bit Kogge-Stone adder

Modern FPGAs contain dedicated addition hardware that is faster than any adder defined using software. Even though most FPGAs just use a simple ripper adder, the connections are short it is optimized at a hardware level for low-latency. In contrast,

building a Kogge-Stone adder out of LUTs and flip-flops results in a lot of resource usage and wide horizontal spans that slow down calculations. Synthesis and implementation for this design were difficult due to Vivado's propensity for optimization and robust detection of concepts such as adders. Flags such as (* dont_touch = "true" *) had to be added to the design to force Vivado to synthesize and implement the Kogge-Stone design. This was difficult as it was hard to understand how synthesis and implementation happened and if the kogge-stone adder design was included.

8.4 Design Four

Design Four is heavily based off of Design One and does aim to rework the majority of calculation/evaluation logic. Rather, the major feature upgrade of this design is the re-design of the instruction load and decode mechanism. A given instruction requires three arguments: the instruction opcode, the polynomial coefficient vector address, and either the polynomial degree specifier N or the buffer amount b (for an EVB instruction). In the software and previous hardware implementations, the main controller reads these arguments sequentially, meaning that three clock cycles are needed to load/decode an instruction. A natural improvement to this mechanism is to pack the three instruction fields into a single instruction and make the PEA responsible for decoding the instruction.

Opcode	**Address**	**Polynomial Degree (N) / Buffer Amount (b)**
Bits [9:8]	Bits [7:5]	Bits [4:0]
Defines the operation to be performed (STP, EVP, etc)	Specifies the vector to be used in the coefficient matrix	Depending on the operation type, provide the relevant value of either N or b

Table 2: Updated PEA instruction format

The updated instruction format is depicted above in Table 2. Each argument combined into a single, efficient 10-bit instruction, as each field only uses its minimum width per its parameters. For example, the address field can take values 0-7 so three bits are used to

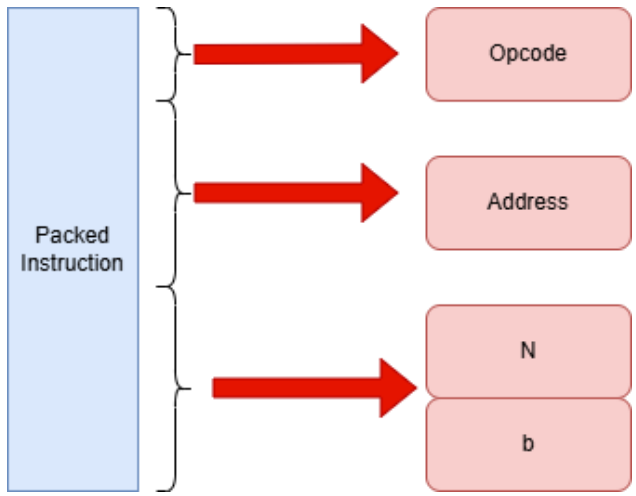

Figure 6: Updated Instruction Decode

encode the address. Since all relevant information can be read in from the control port in a single cycle, the instruction can also be decoded in said cycle. The decoding is performed via simple splitting of the instruction as demonstrated in Figure 6.

It should be noted that both N and b are assigned the same value at the same time. This is not an issue as the PEA will only execute a given instruction with the relevant parameters. For example, if an instruction means for the final field to be treated as N, the PEA will only use N and won't use b despite these two registers having the same value. The register containing b will only be used if a new instruction is read that provides a new, intended value of b.

The main benefit of this re-design is the saving of two clocks in loading/decoding. Although the initial design takes 66 clock cycles to produce a result, removing two cycles is still around a 3% improvement in throughput. Furthermore, for a generic, long processing chain of n instructions,

$$\text{Time saved} = 2 \cdot n \cdot \text{clk_period} \tag{5}$$

This is a significant result, especially for large scale usage with long chains of operations.

In the following case of executing 100 instructions with a 100 MHz clock:

$$\text{Time saved} = 2 \cdot 100 \cdot 10\text{ns} = 2000\text{ns} \tag{6}$$

Another feature of this re-design is that it aims to reduce general logic utilization by removing unnecessary wire lengths as possible (e.g. making a 32-bit wire only 4-bits if only 4 bits are needed).

8.5 Comparison Between Designs

To further analyze and compare these designs, we performed synthesis and implementation using AMD Vivado. Synthesis and implementation were performed targeting the xczu5ev-fbvb900-2-i FPGA part from AMD. Although we did not physically implement/place the design onto the FPGA, all designs are capable of being placed. When synthesis and implementation are ran, Vivado automatically generates reports summarizing the behavior and resource usage of the designs. Specifically, we analyzed the post-implementation utilization, power, and timing reports.

Figure 7 demonstrates the differences in resource utilization by the given designs. The graph compares usage of the AMD FPGA primitives Look-Up-Tables (LUTs)(used to implement combinational logic), Flip-Flops (used to implement sequential logic), DSPs (used to implement specific arithmetic operations)[**?**].

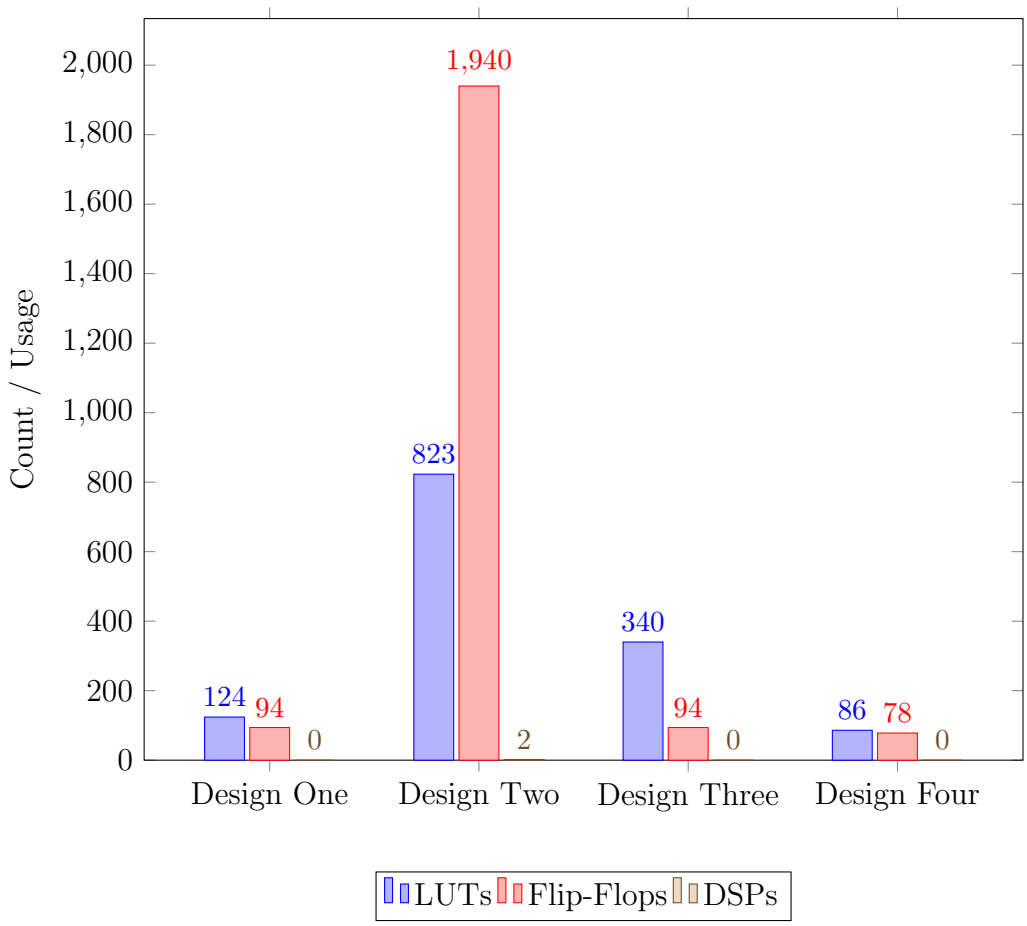

Figure 7: FPGA resource utilization comparison

Designs One and Four appear to be the most resource efficient, while Design Two consumes significantly more of each resource. Design Four is expected to be slightly more efficient that Design One; by simplifying the instruction decode mechanism and shaving some unneeded logic, Design Four is about 20 to 30 percent more efficient than Design One. Since the arithmetic operations in Design Two did not synthesize well compared to the synthesis of standard behavioral arithemtic techniques, it has much higher resource usage, although still small in total utilization as compared to the FPGA part. The same reasoning applies to Design Three: although the Kooge-Stone adder is a highly performant adder, Vivado is able to generate a more resource optimized adder when using behavioral techniques as in Design One.

Figure 8 demonstrates the differences in power consumption by the given designs. Note that power estimates are divided into total power consumption and non-IO power consumption. Since our synthesized/implemented design do not feature any FPGA-specific IO, Vivado has to make certain assumptions and choices about how to route these non-existent IO pins. To ensure that inferences/choices during routing and placement do not affect our analysis, both total power and total power minus IO power are shown. It should also be noted that the power is simply an estimate, especially as power calculations were performed without a system stimulus defined or without strict, specific power constraints (such as the toggle rate or static probability) [**?**].

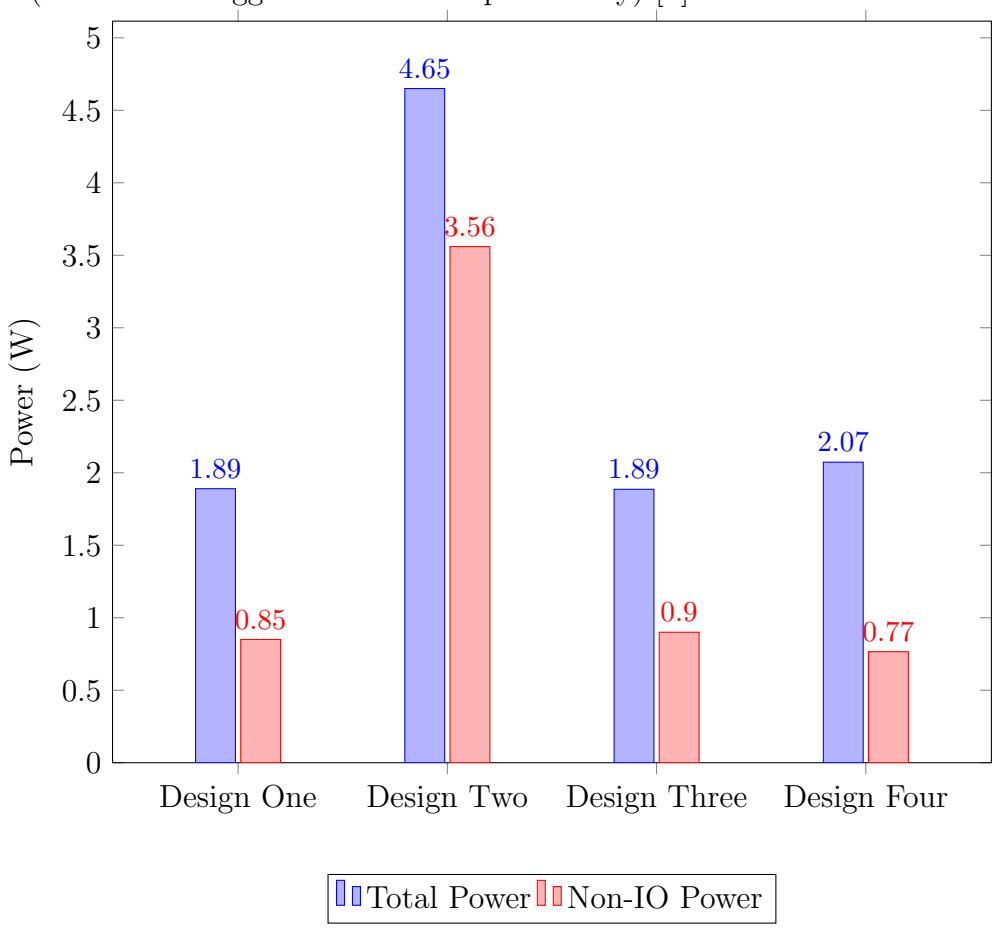

Figure 8: FPGA power consumption comparison

Again, Designs One, Three, and Four have very similar results due to their almost identical designs (the single cycle instruction decode saves some non-IO power while using

more total power). Since Design Two uses way more logic blocks, it consumes significantly more power.

Additionally, we measured the average time of each of the designs and instruction latency shown in Table 3. We ran five benchmark designs and took the average time recorded during the simulation to determine the average benchmark time, then divided the benchmark time by the number of instructions in each benchmark to determine the latency, and took the average of that to get the average instruction latency with overhead. The simulations were run with a 10ns clock, which we used to calculate the average cycles per instruction.

Design	Avg. Benchmark Time (ns)	Avg. Instruction Latency	Avg. Cycles Per Instruction
One	4815	732.78	73.278
Two	2181	328.82	32.882
Three	4815	732.78	73.278
Four	4829	729.71	72.971

Table 3: Performance comparison between designs

Designs One, Three, and Four have very similar times, which Design Four using slightly less cycles per instruction due the the single-cycle decode mechanism. Most significantly, the instruction latency of Design Two is shown to be about half of the other designs, demonstrating the positive effects and speed-up of the Horner evaluation method.

Furthermore, the maximum operating frequency is another useful metric to judge the speed of a design as it represents the maximum clock frequency at which the design can run without timing violations. It can be calculated by using the equation (7), which defines F_{max} as the inverse of the difference between the clock period and the Worst Negative Slack (WNS) [**?**]. In this calculation, WNS is a timing metric used to measure the worst timing violation in the circuit's implementation [**?**]. Negative slack is generally defined as a timing violation while positive slack indicates that signals have sufficient time to arrive to their

destination. Table 4 depicts the WNS and F_{max} values for each design.

$$F_{\max} = \frac{1}{T - \text{WNS}} \tag{7}$$

Design	WNS (ns)	F_{max} (GHz)
One	8.551	0.690131125
Two	5.249	0.210482004
Three	8.536	0.68775790
Four	8.12	0.531914894

Table 4: F_{max} comparison between designs

From here, the calculated Cycles Per Instruction (CPI) and F_{max} can be used to calculate throughput, which is measured in Instructions Per Second (IPS). We used equation (8) to perform these calculations. The results are presented in Table 5.

$$\text{IPS} = F_{\max}/\text{CPI} \tag{8}$$

The result of these calculations are shown in Table 5.

Design	CPI	F_{max} (MHz)	IPS (MIPS)
One	73.278	690.131125	9.417983569
Two	32.882	210.482004	6.401131318
Three	73.278	687.75790	9.3855986
Four	72.971	531.914894	7.289402639

Table 5: Throughput calculations between designs

Design One has the best throughput. Although it has a slightly larger instruction latency than Design Four, its critical path is noticeably shorter (e.g. more WNS), meaning that the high potential speedup in operating frequency dominates the small difference in instruction latency. Despite the fact that Design Two has a much lower CPI, in doing so it has a critical path that is significantly longer than both other designs, resulting in a severely lower maximum operating frequency, and thus the lowest throughput. Design Three has a very similar throughput as Design One, which makes sense since the

implemented Kooge-Stone adder should have similar performance to optimized behavioral synthesis.

8.6 Pareto Analysis

A Pareto optimal solution is a solution in a multi-objective problem that has the best trade-off between all constraints/objectives. Essentially, a Pareto solution cannot be modified to better meet a certain goal without significantly reducing progress towards another [**?**]. In the context of this project, the two objectives being considered are the design performance (instruction throughput) and the utilization/power consumption of the design. Utilization and power design are inherently related (more logic devices means more power consumed) so it makes sense to group them together here. An optimal design in our case is one that maximizes performance while minimizing power consumption and area.

These objectives are compared below in Figures 9 and 10. To combine utilization into a single metric, we used the following equation:

$$\text{Utilization} = Num_{\text{LUT}} + Num_{\text{FF}} + 100 * Num_{\text{DSP}} \tag{9}$$

This equation is by no means a formal or standard definition for utilization; it is simply a metric we better visualize total utilization. The number of LUTs and Flip-Flops both have weights 1 while the number of used DSP primitives is weighted by 100: this was decided because DSPs are a far more scarce resource in FPGA design, especially in the implementation of signal processing or mathematical systems.

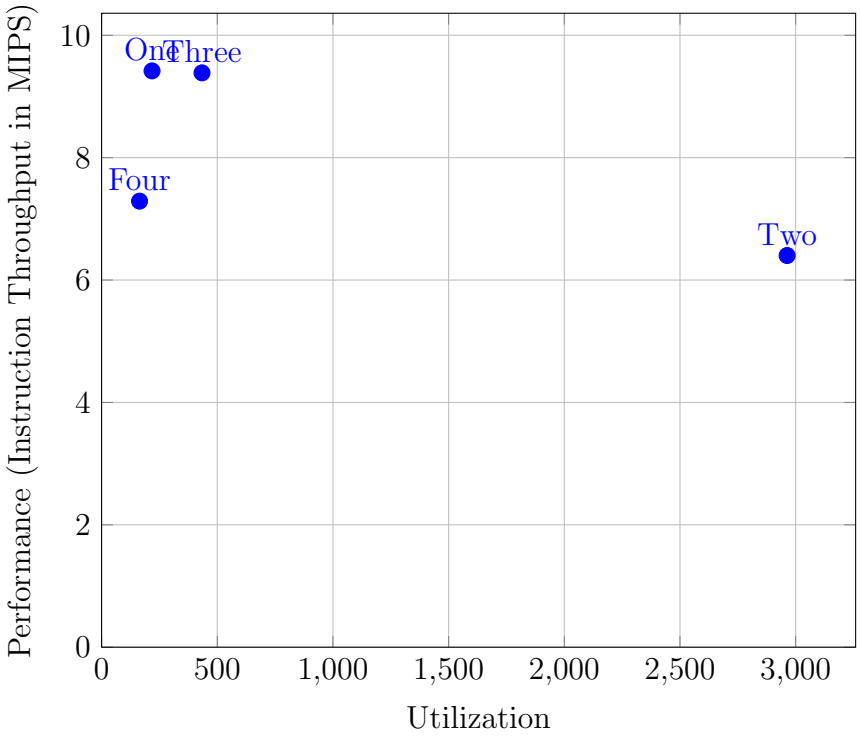

Figure 9: Performance vs. Utilization for All Designs

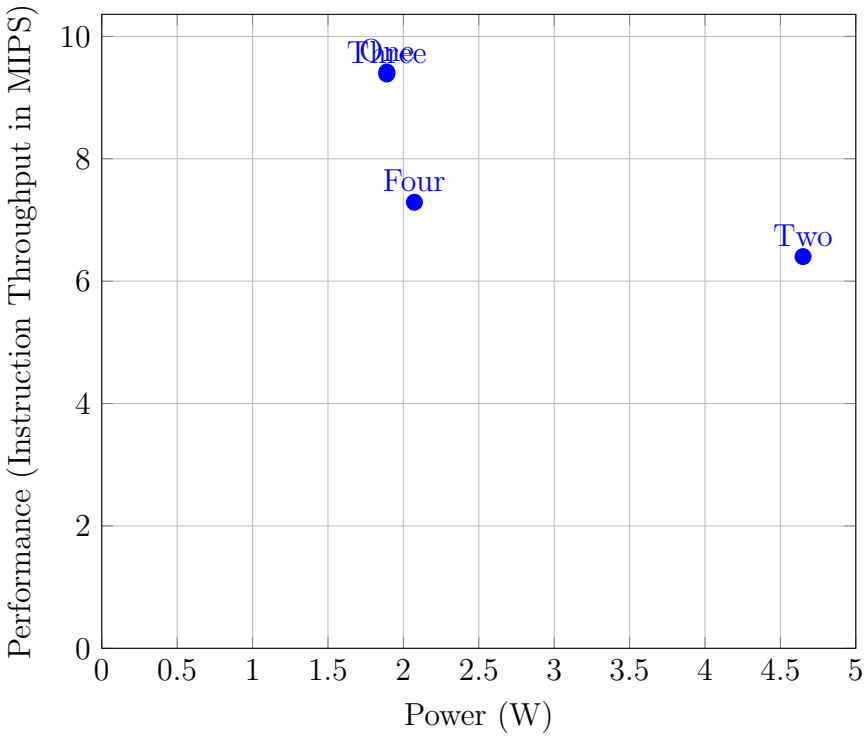

Figure 10: Performance vs. Power for All Designs

In this scenario, since our goal is to maximize performance (y-axis) and minimize power/utilization (x-axis), Pareto optimal solutions will be those closest to the top-left corner of the graphs. In both comparisons, this proves to be Design One; it has the most throughput while also maintaining minimal values for resource and power consumption. This result makes sense. Design Two, The Horner-based polynomial evaluator requires significantly more FPGA resources because it performs a multiply-accumulate operation inside a feedback loop, which prevents the synthesis tool from pipelining or sharing DSP blocks. This forces the multiplier to be implemented with more DSP slices or LUT-based logic, increasing area. Additionally, the design uses a large combinational coefficient selection mux on the multiplier input, further increasing LUT usage and lengthening the critical path. Finally, the feedback structure causes bit-width growth in the intermediate results, requiring larger multipliers. Design Three has performance and power metrics similar to Design One. However, the custom Kooge-Stone adder uses significantly more LUTs as compared to the Vivado optimized behavioral addition in Design One, making Design Three more resource inefficient while not offering better performance or power. Design Four aims to improve Design One by making the instruction decode mechanism a single cycle. While this change results in a decrease in the instruction latency, the design now features a slightly longer critical path, resulting in a lessened maximum operating frequency and thus a lower maximum throughput.

9 Technical Analysis for System and Subsystem

Design_one demonstrates a systematic approach to hardware design that balances functionality, resource utilization, and performance, with the sequential polynomial evaluation method resulting in a latency of approximately 66 clock cycles for a 10th-degree polynomial evaluation. The design complexity is manageable from a maintainability perspective, with the polynomial evaluator maintaining a six-state machine (IDLE,

WAIT_POWER, MULTIPLY, ACCUMULATE, NEXT_TERM, DONE) that coordinates with the power calculator's three-state machine (IDLE, COMPUTE, DONE), providing clear separation between polynomial evaluation logic and power calculation logic. The coefficient memory module implements a dual-port memory structure with separate read and write interfaces, allowing concurrent access for coefficient storage during STP operations and coefficient retrieval during EVP operations, while the design employs signed arithmetic throughout with 16-bit signed coefficients and 32-bit signed results to ensure correct handling of negative values and prevent overflow. Through development and analysis, we learned that while the sequential nature creates a performance bottleneck for higher-degree polynomials, the modular architecture facilitates understanding and maintenance, with error detection implemented at multiple levels through parameter validation in the controller and operation-specific validation in each operation module, providing comprehensive error reporting through the status FIFO interface.

10 Design Validation for System and Subsystems

The benchmark inputs used to evaluate design_one were carefully selected to exercise all code paths and edge cases within the design, including unit tests for each of the four primary operations (STP, EVP, EVB, RST), comprehensive error condition tests, and multi-operation sequence tests that exercise the full state machine behavior. Functional tests verify correct polynomial storage, single evaluation, and batch evaluation operations using a variety of polynomial coefficients and input values, including edge cases such as zero coefficients, negative values, and maximum degree polynomials, while error condition tests validate that design_one correctly handles invalid addresses (addresses ≥ 8), uninitialized coefficient vector access, invalid polynomial degrees ($N > 10$), and invalid batch sizes ($b > 31$), with each error condition producing the appropriate status code. These benchmark inputs are effective at evaluating the efficiency of design_one because

they systematically test the sequential power calculation approach, validating that the polynomial evaluator correctly coordinates with the power calculator across all polynomial degrees from 0 to 10, and specifically target the design_one architecture by testing the interaction between the polynomial evaluator and power calculator modules to verify that the sequential term-by-term evaluation produces correct results, ensuring functional equivalence with the C reference implementation and providing a solid baseline for comparison with alternative design implementations.

11 Test Plan

We designed our unit tests independently of the design of our PEA. While some team members worked on implementing the software and hardware implementations of the PEA, one team member used the design requirements that were described in the project description to design test cases that verified our design fulfilled those minimum requirements. To design the test cases first we came up with a comprehensive set of test vectors that thoroughly tested the minimum and maximum input parameters, as well as different error conditions that had to be checked. Then using those inputs we wrote what we expected to be the correct outputs. This allowed us to shape the design of the PEA around how we wanted it to behave instead of making the tests to fit our implementation, which could cause us to miss potential errors or edge cases. Additionally, we used the same test vectors for both the C and Verilog designs to ensure that all of the designs were subject to the same requirements, some of the verilog designs required the use of the **dxitsout** command to create the correct output file after verifying the correct outputs manually.

Our test suite was organized to allow the user to run the **dxtest** command in the top level directory to test the design implemented in C along with all of our alternate designs implemented in Verilog. First, to run the tests, the C designs must be compiled using the make script and all the Verilog designs must be made in the src directory. Within the test

directory it contains a test_c directory along with test_verilog_x for each of the verilog designs. Within these directories each test was separated by the instruction it was testing. Then, the individual tests for STP,EVP,EVB,RST, and the multi tests were separated into different test directories. This strategy allowed us to test different functions individually as we designed the PEA so we could test the functionality of the STP command before EVP was completed by running **dxtest** only in the test_STP directory. Each of these test directories then contained test directories containing scripts to run the test, correct_outputs, expected_errors, input files, and output files.

The design of our testbench also facilitated a simplified and uniform test procedure. To test the software design we utilized a util directory that contained our driver code setting up our testbench. This code instantiated and connected the Actor and FIFOs, and connected the input and result files that provided data for the FIFOs. The Verilog test directories each contained a testbench that instantiated the actor and FIFOs and initialized the data and command FIFOs within the program instead of using a file, the testbench then printed the outputs of the result and status FIFOs.

12 Project Planning and Management

In this project we initially broke down the tasks into the Verilog, C programming, and unit testing components, and then added on the responsibility of alternate designs as shown in the Gantt chart in Figure 11. All group members collaborated on planning the overall architecture of the PEA, selecting the states and handling the error modes. Two team members began working on the C coding portion first, and simultaneously one member began working on creating the unit tests. Both of these tasks finished around the same time allowing us to begin verifying the C code using the unit tests. The development of the Verilog design began slightly after the C portion, since the functionality of the hardware design was modeled after the software design, but was continued to develop

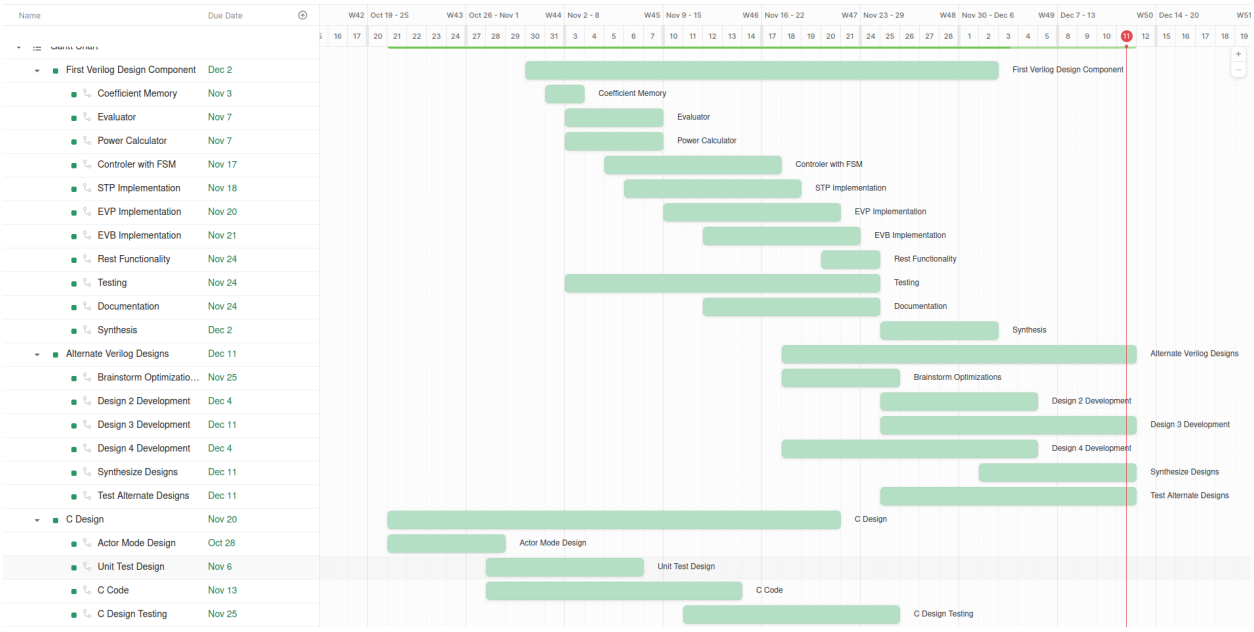

Figure 11: Gantt Chart

independently. The Verilog design was finished not long after testing the C portion, allowing us to implement the same test suite for the Verilog design and compare outputs. With the completion of the C code and testing, three of our team members were able to focus their efforts on creating alternate designs, first brainstorming possible ways to optimize the design and then implementing them in Verilog. We then tested and synthesized the alternate designs to compare to the original design.

13 Conclusion

The overall goal of this project was to design and implement a Polynomial Evaluation Accelerator (PEA) actor in both software (C) and hardware (Verilog). Our main approach was to create a general, actor design and then implement and validate said design in both domains. Once we completed our initial designs, we created three alternative Verilog designs testing the usage of different evaluation methods or adders. Overall, the project was successful: all five designs work as intended and we have a clear Pareto (best) hardware design. This design ended up being the original Verilog design, which had a good

performance/utilization tradeoff due to its sequential nature and usage of behavioral Verilog, which Vivado is very good an interpreting/optimizing. The only major hiccup we had was through using source control/Git. Due to some limited experience among team members, it took us a while to best decide how to use various source control features (branches, .gitignore, etc). Our main solution here was a couple large refactors of the codebase to reset our repository. Besides this, the project was smooth and we gained valuable experience in both designing and implementing a digital system from concept to product.